

PawPlate C Coding Format Guideline

Purpose

This coding style guide aims to:

- Establish a consistent C coding standard across the PawPlate project.
- Improve code readability through uniform formatting and naming conventions.
- Increase code maintainability by making modules easier to understand and modify.
- Reduce programming errors caused by inconsistent coding practices.
- Simplify code reviews and debugging
- Make it easier for new developers to understand and contribute to the codebase.
- Encourage professional software engineering practices throughout the project.
- Present a clean, organized, and professional codebase that reflects well during project demonstrations, design reviews, and future **technical interviews**.

1. Indentation and Brackets

Use **2 spaces** for indentation. Do not use tabs.

Opening braces must be placed on a new line for functions, structs, loops, conditionals, and switch statements.

Braces are optional if it is only followed by 1 statement.

Acceptable:

```
// Comment
if (Condition)
    DoSomething();
else
    DoSomethingElse();
```

```
// Comment
if (Condition)
{
    DoSomething();
    DoSomething2();
}
else
{
    DoSomethingElse();
    DoSomethingElse2();
}
```

Not Acceptable:

```
if (Condition)
{
    DoSomething1();
    DoSomething2();
}
else
    DoSomethingElse();
```

2. Naming Style

Use **double camel cases** for everything except macros and constants.

2.1 Variables

Use **Hungarian-style prefixes** for variables to show type and intent.

Prefixes	Type
uc, c,	uint8_t or char, int8_t,
us, s	uint16_t, int16_t
ul, l	uint32_t, int32_t
ull, ll	uint64_t, int64_t
u	unsigned
b	bool
f	float
d	double
p	pointer
a	array
st	struct
e	enum

Example combinations:

aul // array of uint32_t

puc // uint8_t pointer

pauc // pointer to array of uint8_t

pst // pointer to structure

past // pointer to array of structure

Example variables:

```
uint16_t usLen
```

```
const char *paucTitle
```

```
static VcpDebugLevelTypeDef ulTheDebugLevel;
```

```
static char aucTheVcpCommandBuf[VCP_COMMAND_BUF_SIZE];
```

2.2 Function and Macro Arguments

Function and macro argument names must end with _.

Function Example:

```
void HandleAssertFail(uint8_t *pucFile_, uint32_t ulLine_)
```

Macro Example:

```
#define ASSERT(expr_) \
do \
{ \
    expr_ ? (void)0 : HandleAssertFail(__FILE__, __LINE__); \
} while (0)
```

2.3 Macros and Constants

Use **SCREAMING_SNAKE_CASE** names for constants and macros.

Example:

```
#define DBG_MASK_VCP (1UL << 31)

#define FAULT_BACKUP_MAGIC_NUMBER (0xFA17FA17U)

#define IS_MEMORY_FAULT_ADDRESS_VALID() \
(((FAULT_BKPR->CFSR & SCB_CFSR_MMARVALID_Msk) != 0U))
```

2.4 Global Variables

Reduce the use of global variables, if we do, naming must start with word “**The**”.

Example:

```
static VcpDebugLevelTypeDef ulTheDebugLevel;

static char aucTheVcpCommandBuf[VCP_COMMAND_BUF_SIZE];
```

2.3 User Defined Data Types

Names for struct, enum, union etc. should end with “**TypeDef**”.

Example:

```
typedef struct
{
    //CPU registers at the time of fault
    __IO uint32_t R0;
    __IO uint32_t R1;
    __IO uint32_t R2;
    __IO uint32_t R3;
} FaultBackupRegsTypeDef;
```

3. Comments

3.1 Control Statement Comments

Every control statement should have a short comment explaining its purpose, especially **if**, **for**, **while**, **do while**, and **switch**.

Examples:

```
// Check if the assert expression failed
if (expr_ == false)
    HandleAssertFail(__FILE__, __LINE__);

// Process all pending bytes in the RX buffer
while (ucHasData == true)
    ProcessRxByte();

// Check for valid debug level and return corresponding string.
switch (eLevel_)
{
    case LEVEL_ERROR:
        return "ERROR";

    case LEVEL_WARN:
        return "WARN ";

    default:
        return "UNKNOWN";
}
```

3.2 Doxygen Comments

Use Doxygen comments for functions, important types, and important macros.

```
/**
 * @brief
 *
 * @param[in]
 * @param[out]
 *
 * @return
 */
```

4. File Layout

Use the existing template ...\\Doc\\Miscellaneous\\codetemplates.xml

Header files should follow this section order:

- Includes

- Public Macros

- Public Type Definitions

- Public Constants

- Public Function Prototypes

Source files should have follow this section order:

- Includes

- Private Macros

- Private Type Definitions

- Private Variables Constants

- Private Function Prototypes

- Public Function Prototypes

- Private Function Prototypes